Laboratorio 3 — Conjuntos de réplicas en MongoDB

TI-4601 Bases de Datos Avanzadas · Grupo 3 · Fabricio Mena · Christopher Rodriguez · Alisson Redondo

1. Configuración del conjunto (pasos 0 a 3)

Máquina de los pasos 0 a 6. Laptop AMD Ryzen 7 5700U, 16 GB de RAM (4 CPU y 9,7 GiB para WSL 2), Windows 11 Home con Ubuntu 24.04 en WSL 2, Docker Desktop 29.5.2 y MongoDB 8.0.34. Registro con hora y `OK` de cada paso en `evidence/comandos.md`.

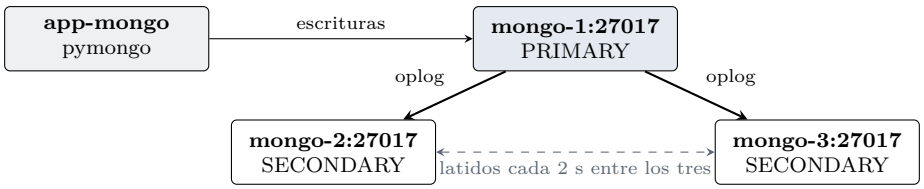
Paso 1: Compose. Los tres nodos arrancan con el mismo comando:

```
mongod --replSet rs-lab3 --bind_ip_all --wiredTigerCacheSizeGB 0.25
```

Servicio	--replSet	hostname	Volumen	¿Publica puertos?
mongo-1	rs-lab3	mongo-1	mongo1	no
mongo-2	rs-lab3	mongo-2	mongo2	no
mongo-3	rs-lab3	mongo-3	mongo3	no

Tras `make lab3-up` los tres contenedores quedaron `healthy` y `make lab3-status` respondió `NotYetInitialized` (`cluster-start.txt`). `--replSet` solo dice a qué conjunto puede pertenecer cada proceso; quiénes son los miembros y quién vota se define con `rs.initiate()`, y hasta entonces no hay conjunto. Por eso el `_id` de la configuración tiene que ser `rs-lab3`: `mongod` rechaza una configuración con otro nombre. Los miembros se nombran `mongo-1:27017` y no `localhost:27017` porque la configuración se copia a los tres nodos y cada uno la usa para llegar a los otros: dentro de un contenedor `localhost` es él mismo, y los nombres `mongo-N` los resuelve el DNS de la red de Compose. Si se borran los volúmenes se pierden los datos y también la configuración del conjunto, que cada nodo guarda en su base `local`; habría que repetir desde el paso 1.

Paso 2: `rs.initiate()`.




```
rs.initiate({
  _id: "rs-lab3",
  members: [
    { _id: 0, host: "mongo-1:27017" },
    { _id: 1, host: "mongo-2:27017" },
    { _id: 2, host: "mongo-3:27017" }
  ]
})
```

`_id: "rs-lab3"` es el nombre del conjunto y debe ser igual al `--replSet`. `members` es la lista de miembros: en cada uno, `_id` es su número dentro del conjunto (0, 1 y 2) y `host` es el nombre y puerto con el que los demás lo alcanzan. Respondió `{ ok: 1 }` y en unos segundos quedaron `mongo-1` como `PRIMARY` y `mongo-2` y `mongo-3` como `SECONDARY`. Se usa un número impar de votantes porque la mayoría es $\lfloor n/2 \rfloor + 1$: con 3 votantes es 2 y se tolera una caída; con 4 la mayoría es 3 y se sigue tolerando solo una, así que el cuarto nodo no agrega tolerancia y en una partición de 2 contra 2 nadie tendría mayoría. MongoDB completó solo los campos `arbiterOnly: false`, `buildIndexes: true`, `hidden: false`, `priority: 1`, `tags: {}`, `secondaryDelaySecs: 0` y `votes: 1` de cada miembro, y los `settings` del conjunto (`rs-inspect.txt`). Con `localhost` cada nodo intentaría conectarse a sí mismo en vez de a los otros, y el conjunto no se formaría.

Paso 3: lectura de `rs.status()` y `rs.conf()` (`rs-inspect.txt`). El primario es `mongo-1`, y `mongo-2` y `mongo-3` copian de él (`syncSourceHost: mongo-1:27017`). El `term` es 1, porque en un conjunto nuevo solo hubo una elección. Los tres tienen `votes: 1` y `priority: 1`. En `settings`, `heartbeatIntervalMillis` es 2 000 y `electionTimeoutMillis` es 10 000: cada nodo manda un latido cada 2 s, y si un secundario pasa 10 s sin saber del primario (unos cinco latidos perdidos) pide votos. `db.hello()` en `mongo-1` da `setName: rs-lab3`, `isWritablePrimary: true` y `secondary: false`.

Entre que el primario cae y un secundario pide votos pasan como máximo unos 10 s: ese plazo es `electionTimeoutMillis`. El `term` equivale al término de Raft del Lab 1: sube en cada elección, y un primario que ve un `term` mayor deja de serlo. `priority` decide entre candidatos que están al día: gana preferencia el de mayor prioridad, y uno con `priority: 0` nunca será primario. Aquí los tres tienen 1, así que cualquiera puede ganar.

2. Datos, carga e índices (pasos 4 y 5)

Predicción (`evidence/predicciones.md`, escrita antes de correr): cada documento quedaría en los 3 nodos, y 2 (el primario y un secundario) deberían tenerlo antes de que `insert_many` responda, porque la conexión usa `w: "majority"` y la mayoría de 3 es 2. Se cumplió: `generar_datos.py` produjo 400 eventos, 1 189 zonas y 162 810 de capacidad, con SHA-256 `9fc6a2db...774a` (`datos-manifest.txt`); `cargar.py` dio sus cinco líneas `OK:` y `make lab3-check` terminó en `6/6`, con diferencia máxima de *optime* entre miembros de 0,0 s (`config-check.txt`).

TODO 1. `cargar_eventos` lee el JSONL línea por línea, junta lotes de 500 y los inserta con `insert_many(..., ordered=False)`, conservando el `_id` de cada evento. Con `ordered=False`, si un documento falla (por ejemplo, un `_id` repetido) el servidor sigue con el resto del lote y reporta los errores al final; con `ordered=True` se detendría en el primero y dejaría el lote a medias. Cada `insert_many` espera a que 2 de los 3 nodos tengan el lote; con 400 eventos fue una sola llamada.

TODO 2. Los índices son `{categoria: 1, fecha: 1}` en `eventos` y `{evento_id: 1, estado: 1}` en `ordenes`: primero el campo de igualdad y después el de rango u orden. Con `categoria` primero, los eventos de una categoría quedan juntos en el índice y ya ordenados por `fecha`, así que el rango `fecha >= d` es un solo tramo y el `sort` sale del índice sin ordenar en memoria.

Consulta: `categoria: "concierto", fecha >= "2026-12-01"`, ordenada por fecha.

	Con índice	**Sin índice** (`hint $natural`)
Etapas del plan	`IXSCAN categoria_1_fecha_1` → `FETCH`	`COLLSCAN` → `SORT`
`totalKeysExamined`	67	0
`totalDocsExamined`	67	400
`nReturned`	67	67
`executionTimeMillis`	1	1

Las dos dan los mismos 67 eventos (`explain-indices.txt`). Con el índice se examinaron solo esos 67 documentos y no hubo etapa `SORT`; sin índice se recorrieron los 400 y hubo que ordenar en memoria. El tiempo empata en 1 ms porque 400 documentos son pocos, pero los documentos examinados sin índice crecen con la colección y con el índice crecen solo con el resultado.

3. Replicación observada (pasos 6 y 7)

Paso 6: oplog (`oplog.txt`). La orden `ORD-MANUAL-1` insertada en el primario aparece en `local.oplog.rs` con `op: 'i'` (insert), `ns: 'boleteria.ordenes'` (base y colección), `o` (el documento insertado), `ts: Timestamp({ t: 1792700740, i: 1 })` (segundo y contador dentro de ese segundo, que ordenan las operaciones) y `t: 1` (el `term` del primario que la escribió). Los dos secundarios estaban a 0 s del primario y `mongo-2` ya podía leer la orden.

El oplog es una colección capada: tiene tamaño fijo (aquí 47 287 MB configurados) y cuando se llena sobrescribe las entradas más viejas, para que la replicación no crezca sin límite. La ventana (`log length start to end`) era de 151 s porque el conjunto era nuevo. Si un secundario se atrasa más que la ventana, las entradas que le faltan ya se borraron, no puede ponerse al día con el oplog y necesita un *initial sync*, que copia otra vez todos los datos.

Al escribir en `mongo-2` la respuesta fue `NotWritablePrimary (code 10107): not primary`: solo el primario acepta escrituras y los secundarios solo aplican lo que trae el oplog. El cliente tiene que encontrar al primario, y por eso la URI del driver lista los tres miembros.

Paso 7: lecturas en secundario (`lectura-secundario.csv`, `lectura-secundario.txt`). Los pasos 7 a 9 se repitieron el 1/11/2026 en la máquina de los pasos 10 a 13: Intel Core i7-6500U (4 hilos), 11 GiB de RAM, Ubuntu 24.04.5, Docker 29.9.2 sin límites de CPU ni RAM y MongoDB 7.0.45, con el disco libre y solo Neo4j del Lab 4 inactivo en segundo plano (`entorno-pasos-7-13.txt`). Las corridas anteriores quedan en el historial de git: la del 23/10 no declaró la RAM y otra del 1/11 se descartó porque coincidió con un *build* de Docker que saturaba el disco. Hubo 500 intentos por modalidad. **Expectativa teórica** (*no se registró una predicción antes de la primera corrida; esta se escribió el 24/10, antes de esta repetición pero con aquellos resultados ya conocidos, y no es una predicción previa*): con `w1_secundario`, casi todas las lecturas obsoletas, porque el primario confirma sin esperar a nadie y la lectura llega al secundario antes de que aplique el oplog; con `majority_secundario`, muchas menos pero no cero, porque al confirmar ya hay un secundario con el dato y la lectura puede caer en el otro; con `causal_majority`, ninguna, porque la sesión causal hace que el secundario espere hasta haber visto la escritura. Los resultados respetaron el orden, aunque `majority_secundario` dio cero en esta corrida:

Modalidad	**Intentos**	**Obsoletas**	**Porcentaje**
`w1_secundario`	500	465	93,0 %
`majority_secundario`	500	0	0,0 %
`causal_majority`	500	0	0,0 %

Las lecturas obsoletas fueron 465/500 con `w:1`, pues el primario no espera replicación; 0/500 con `majority`, que no es

una garantía: al confirmar basta que dos nodos tengan el dato, y la lectura puede ir al secundario atrasado (la corrida del 23/10 tuvo 3/500); y 0/500 con sesión causal y garantías `majority`, que permite observar escrituras previas de esa sesión. Esto no garantiza frescura para lecturas arbitrarias desde un secundario.

CAP/PACELC por modo. `w1_secundario` prioriza disponibilidad y baja latencia y sacrifica consistencia: el primario confirma sin quórum y, ante una falla, esa escritura puede revertirse. `majority_secundario` prioriza consistencia en la escritura, que sin quórum no se confirma (CP) y sin partición cuesta latencia (PACELC), pero la lectura en un secundario aún puede salir obsoleta. `causal_majority` prioriza consistencia dentro de la sesión: el secundario espera hasta alcanzar la escritura, y sacrifica latencia y disponibilidad de la lectura si ese secundario no avanza. Para "mis entradas" se prefiere lectura causal o del primario; "eventos destacados" puede tolerar secundarios atrasados. Los tres nodos comparten una computadora: estos porcentajes no se extrapolan a una red distribuida real.

4. Atomicidad y costo de las garantías (pasos 8 y 9)

Paso 8: compra atómica (`comprar.txt` y `comprar-roto.txt`). El **TODO 3** registra cada orden con `_id` único y estado `pendiente`; ante `DuplicateKeyError` devuelve el estado ya guardado y no vuelve a descontar. El stock se actualiza con un solo `update_one`: `$elemMatch` exige *en el mismo elemento* del arreglo `zonas` el nombre pedido y `disponibles >= cantidad`; `$inc` sobre `zonas.$.disponibles` efectúa el descuento atómico. Dos condiciones independientes, `zonas.nombre` y `zonas.disponibles`, pueden satisfacerse en zonas distintas. Con `zonas: [{nombre: "General", disponibles: 0}, {nombre: "VIP", disponibles: 10}]` y una compra de 2 en General, las condiciones separadas hacen match (General aporta el nombre y VIP las entradas) y el `$` no garantiza apuntar a General, así que se vende sin stock; con `$elemMatch` no hay match y la orden queda `agotada`. Dos `update_one` simultáneos al mismo documento se serializan: cada uno evalúa su filtro y aplica su cambio sin intercalarse con el otro, y no se pierde ningún `$inc`. Sobre documentos distintos no hay esa garantía: sin transacción, otro cliente puede ver uno cambiado y el otro no, y una falla puede dejar solo uno aplicado.

Resultado (200 compradores; 120 entradas)	Correcta	Rota
Órdenes confirmadas / agotadas	120 / 80	200 / 0
Entradas General restantes	0	110
VIP: compras confirmadas de 4 sobre 10	2	2

La versión rota separó `find_one` y `update_one` con `$set`: varias solicitudes leyeron el mismo saldo y sobrescribieron las actualizaciones de otras (*lost updates*). La carrera depende del intercalado: esta vez no se manifestó en VIP, con solo 3 compradores, y sí en la corrida del 23/10 (3 confirmadas). En cambio, la versión atómica pasó las seis verificaciones, incluida la idempotencia (cinco reintentos de la misma orden, un solo descuento). La atomicidad cubre *un documento*, no toda la compra: si el proceso cae entre descontar el evento y marcar la orden `confirmada`, puede quedar `pendiente` con stock ya reducido. Se detectarían órdenes pendientes antiguas para reconciliarlas; no se debe reejecutar ciegamente el descuento. Con la zona sin compras en curso, se compara `capacidad − disponibles` con la suma de órdenes confirmadas: si la diferencia incluye a la pendiente, ya descontó y se marca `confirmada`; si no, se marca `agotada`. La garantía del descuento es **como máximo una vez**: reintentar con el mismo `orden_id` no descuenta de nuevo, pero sin reconciliación la orden puede quedar a medias, así que no es «exactamente una vez».

Paso 9: costo de *write concern* (`write-concern.csv`, `write-concern.txt`). El **TODO 4** usa `with_options` y `WriteConcern` para configurar la garantía; mide cada `insert_one` con `perf_counter`, sin incluir la creación del cliente en la latencia. Hubo 30 inserciones de calentamiento y 300 muestras por variante (1 200 observaciones; CSV de 1 201 líneas).

Garantía	p50 (ms)	p95 (ms)	p99 (ms)	ops/s
`w1`	0,94	1,58	3,25	941
`w1_journal`	16,20	58,41	125,87	42
`majority`	15,31	37,05	58,25	54
`w3`	3,22	9,19	12,11	238

La predicción, anotada antes de la primera corrida (`comandos.md`), ordenaba `w1`, `w1_journal`, `majority` y `w3` (quizá la más lenta), con `majority` en torno a 1,5–3 veces `w1` y diferencias pequeñas. Por p50 solo se cumplió que `w1` es la más rápida: el orden fue `w1`, `w3`, `majority` y `w1_journal`, y `majority` resultó 16,3 veces `w1`. `w1` espera solo a que el primario aplique la escritura. `j:true` agrega la espera a que quede en el journal en disco, de modo que sobrevive a un reinicio del proceso. `majority` espera además a un secundario y, como `writeConcernMajorityJournalDefault` es `true` en este conjunto, también exige journal. `w3` espera a los dos secundarios, pero sin pedir journal. Los datos forman dos grupos: sin journal, `w1` (0,94 ms) y `w3` (3,22 ms); con journal, `majority` (15,31 ms) y `w1_journal` (16,20 ms). Esto indica que aquí el costo dominante es el disco, un SSD SATA compartido por los tres nodos, y no la replicación: en una sola máquina los secundarios aplican casi al mismo tiempo. Por eso `w3` supera a `majority` aunque espera a más nodos. Es una sola corrida con variantes medidas sucesivamente, así que el empate entre las dos con journal no permite ordenarlas. Con nodos en regiones distintas, `majority` sumaría a cada escritura el viaje de ida

y vuelta al secundario más cercano y `w3` al más lejano, mientras que `w1` (y `w1_journal`) no cambiaría, porque no espera a ninguna réplica. Mediciones realizadas en la máquina descrita arriba, bajo contenedores locales (sin latencia de red interregional); `docker info` reporta 4 CPU y 11,6 GiB, sin límites por contenedor.

Decisiones de los cinco TODO. En **TODO 1**, `ordered=False` permite continuar el lote tras errores individuales; en **TODO 2**, el índice compuesto ordena igualdad antes de rango; **TODO 3** usa descuento condicionado y atómico; **TODO 4** compara latencias por garantía; **TODO 5**, clasifica escrituras confirmadas, inciertas y pérdidas para estimar RPO, sin confundir error del cliente con ausencia de escritura.

5. Fallas (pasos 10 a 13)

Máquina de los pasos 7 a 13. Intel Core i7-6500U (4 hilos), 11 GiB de RAM, Ubuntu 24.04.5 (kernel 7.0), Docker 29.9.2 sin límites de CPU ni RAM y `--wiredTigerCacheSizeGB 0.25` por nodo. **Se usó MongoDB 7.0.45 y no 8.0:** `mongo:8.0` no arranca en kernels 6.19 o más nuevos (SERVER-121912, TCMalloc y `rseq`), así que un `docker-compose.override.yml` local (no versionado) cambió solo la imagen. Replicación, elecciones, `w:majority`, `electionTimeoutMillis` y el *step down* funcionan igual en 7.0.

Las expectativas de las tres fallas son de Christopher Rodriguez (`predicciones.md`). Se escribieron el 24/10, a partir de la teoría y después de ejecutar las fallas el 23/10, por lo que no son predicciones registradas previamente.

TODO 5: cómo se clasifica el RPO. `calcular_rpo` solo usa operaciones entre conjuntos. `presentes` se lee con `readConcern: "majority"`: solo cuenta lo que tiene una mayoría y, por tanto, no se puede revertir en un *rollback*. Autotest: 3/3 `OK`; línea base (`rpo-baseline.txt`): 20 intentos, 20 confirmados, 20 presentes, `PERDIDOS 0`.

Clave	Fórmula	Significado
`perdidos`	`confirmados − presentes`	se confirmaron y no están: RPO > 0
`desconocidos_presentes`	`sin_confirmar ∩ presentes`	dieron error pero **sí** quedaron escritas
`desconocidos_ausentes`	`sin_confirmar − presentes`	dieron error y no quedaron
`inesperados`	`presentes − confirmados − sin_confirmar`	nunca se intentaron (error de la prueba)

Escenario	Caídos	Intentos	ok	error	Tipos de error	RTO	RPO
A, secundario	mongo-2	80	80	0	—	sin corte	0
B, primario	mongo-1	104	101	3	`ServerSelectionTimeout` ×3	10,01 s	0
B5, `electionTimeoutMillis=5000`	mongo-3	113	112	1	`ServerSelectionTimeout` ×1	4,99 s	0
C, sin mayoría	mongo-1 y 2	109	95	14	`WTimeout` ×3, `WriteConcern` ×1, `ServerSelectionTimeout` ×10	≈34 s*	0

* Medido desde la señal, que se escribió tarde (ver Falla C). Entre la última y la primera escritura confirmada pasaron **43,25 s** (1792818002.918506 − 1792817959.666162). En C el primario vuelve cuando se reinician los nodos caídos, no por una elección automática: ese «RTO» depende del operador (`sleep 20 + 10` del guion), no de MongoDB.

5.1. Falla A: cae un secundario

Expectativa teórica: sin errores, porque quedan 2 de 3 votos (mayoría, pero sin margen); p50 parecida y p99 algo mayor, porque ahora hay que esperar justo al secundario que queda, y quizá un pico en la escritura en curso; RPO 0 y vuelta del nodo como SECONDARY sin *initial sync*.

Precondición (`falla-a-before.txt`): mongo-1 PRIMARY, mongo-2 y mongo-3 SECONDARY. **Falla:** `docker stop --timeout 0 ti4601-mongo-2`, señal `1792817808.142715757` (2026-10-23T22:56:48.14-06:00, `falla-a-stop.txt`). **Durante** (`falla-a-status.txt`): mongo-1 PRIMARY, mongo-2 `not reachable/healthy`, mongo-3 SECONDARY. Filas de `falla-a.csv` (phase, seq, status, latency_ms, completed_epoch):

```
antes-falla,12,ok,3.9,1792817807.919221
despues-falla,13,ok,5.2,1792817808.420788   <- 278 ms después de la señal
despues-falla,14,ok,4.7,1792817808.920558
```

Ningún error y ningún cambio apreciable de latencia: p50 de 5,3 ms antes (n = 13) y 4,7 ms después (n = 67); p99 de 14,6 y 8,4 ms. Con `w:majority` el primario necesita su propio *ack* más el de **un** secundario; con mongo-2 caído quedan mongo-1 y mongo-3, 2 de 3 votos: hay mayoría para mantener al primario y para confirmar escrituras. **Contra la expectativa:** se cumplió que no habría errores y que la p50 sería parecida; no subió la p99 (bajó de 14,6 a 8,4 ms) ni hubo pico en la escritura en curso (seq 13, 5,2 ms): con los tres nodos en la misma máquina, los dos secundarios responden casi al mismo tiempo, así que esperar justo a mongo-3 no cuesta más que esperar al más rápido. Al volver (`falla-a-recovered.txt`), mongo-2 entra como SECONDARY y retoma la replicación desde su último `optime` leyendo el oplog del primario; no necesita *initial sync* porque estuvo fuera unos 15 s, muy por debajo de la ventana del oplog. `falla-a-rpo.txt`: 80/80 presentes, **PERDIDOS 0**.

5.2. Falla B: cae el primario

Expectativa teórica: unos 11 a 13 s sin primario: 10 s de `electionTimeoutMillis` sin latidos, más la votación y el descubrimiento del driver (~1 s cada uno); con `electionTimeoutMillis = 5000`, de 6 a 8 s. Depende sobre todo de `electionTimeoutMillis`. La sonda vería 3 o 4 `ServerSelectionTimeoutError` seguidos, de unos 3 s cada uno, que `retryWrites` no oculta porque reintenta una sola vez, y después una escritura lenta. El `term` subiría en 1, el viejo primario volvería como SECONDARY y el RPO sobre lo confirmado sería 0.

Precondición (`falla-b-before.txt`): mongo-1 PRIMARY, `term=1`, `electionTimeoutMillis=10000`. **Falla:** `docker kill ti4601-mongo-1`, señal `1792817872.249831348` (2026-10-23T22:57:52.27-06:00). **Durante** (`falla-b-status.txt`, 15 s después): mongo-1 caído, mongo-2 SECONDARY, **mongo-3 PRIMARY, `term=2`**. Filas de `falla-b.csv`:

```
antes-falla,11,ok,6.7,1792817871.591525      <- última confirmada antes de la falla
antes-falla,12,error,3009.0,1792817875.095516,ServerSelectionTimeoutError
despues-falla,13,error,3001.2,1792817878.097063,ServerSelectionTimeoutError
despues-falla,14,error,3000.9,1792817881.098176,ServerSelectionTimeoutError
despues-falla,15,ok,1161.6,1792817882.260059 <- primera confirmada tras la señal
```

RTO = 1792817882.260059 − 1792817872.249831 = **10,01 s**. El seq 12 figura como `antes-falla` porque la sonda lo envió con el `kill` en curso: la señal se escribe cuando `docker kill` devuelve, unos cientos de ms después.

Por qué el RTO supera `electionTimeoutMillis`. Los secundarios dejan de ver los latidos del primario (cada 2 s) y esperan `electionTimeoutMillis` (10 s, con una pequeña variación aleatoria) antes de convocar la elección; luego viene la votación, con un `term` nuevo; después el driver descubre al nuevo primario (`heartbeatFrequencyMS=1000` en la sonda), y por último se reintenta la operación pendiente: el seq 15 tardó 1 161,6 ms porque se emitió durante la elección. **Errores y `retryWrites`:** el driver reintenta **una sola vez** y solo si encuentra un primario dentro de `serverSelectionTimeoutMS=3000`; sin primario durante unos 10 s, tres intentos seguidos agotaron los 3 s y salieron como `ServerSelectionTimeoutError`. El reintento oculta fallas cortas, no una ventana mayor que ese plazo. **`term` y el viejo primario:** el `term` pasó de 1 a 2; al volver, mongo-1 ve un `term` mayor y entra como SECONDARY (`falla-b-recovered.txt`): todos tienen `priority` 1, mongo-3 está sano y un nodo con `term` viejo no gana elecciones ni acepta escrituras. **RPO** (`falla-b-rpo.txt`): 101 confirmadas, 101 presentes, **PERDIDOS 0**; los 3 errores no quedaron escritos. Con `w:majority` cada `ok` ya estaba en 2 de 3 nodos, y lo tenía al menos uno de los que eligieron al nuevo primario.

B5, opcional: `electionTimeoutMillis = 5000` (`falla-b5-*`). Precondición: mongo-3 PRIMARY, `term=3`; tras `docker kill ti4601-mongo-3` el nuevo primario es mongo-2, con `term=4`; al final se restauró el valor a 10 000 (`falla-b5-recovered.txt`).

```
antes-falla,12,ok,4.7,1792818096.629883
despues-falla,13,error,3095.6,1792818100.221038,ServerSelectionTimeoutError
despues-falla,14,ok,1593.0,1792818101.814277   <- primera confirmada
```

Con la señal en `1792818096.827114`, **RTO = 4,99 s**, frente a 10,01 s con el valor por defecto. El RTO escala casi 1:1 con `electionTimeoutMillis`, que es el término dominante; bajarlo acorta el corte a cambio de más elecciones innecesarias si la red tiene picos de latencia. RPO 0, 1 error no escrito. **Contra la expectativa:** los dos RTO quedaron por debajo de lo estimado (10,01 s frente a 11–13 s y 4,99 s frente a 6–8 s): la votación y el descubrimiento del driver tardaron menos de lo supuesto, y `electionTimeoutMillis` explica casi todo el corte. Los errores, el cambio de `term` y el RPO 0 coincidieron.

5.3. Falla C: dos secundarios caídos, primario vivo (sin mayoría)

Expectativa teórica: no habría primario: el sobreviviente ve 1 de 3 votos y hace *step down* pasado `electionTimeoutMillis`. Cada insert `w:majority` pasaría por dos fases: mientras el nodo sigue siendo primario, se aplica localmente y devuelve `WTimeoutError` a los 2 s, con resultado desconocido; después del *step down*, `ServerSelectionTimeoutError` cada ~3 s sin escribir nada. No habría escrituras confirmadas hasta que vuelvan los nodos, y al volver aparecerían errores que sí se escribieron. Frente a la Falla B del Lab 1, los dos priorizan la consistencia (CP); en Raft el líder no aplica la escritura hasta tener quórum, mientras que MongoDB la aplica primero y espera la mayoría después.

Precondición (`falla-c-before.txt`): mongo-3 PRIMARY, mongo-1 y mongo-2 SECONDARY; sonda con `--w majority --wtimeout-ms 2000`. **Falla:** `docker kill ti4601-mongo-1 ti4601-mongo-2`, señal `1792817968.942381` (2026-10-23T22:59:28.94-06:00). **Durante** (`falla-c-status.txt`, 20 s después): mongo-1 y mongo-2 caídos y **mongo-3 SECONDARY**: no queda primario. Filas de `falla-c.csv`:

```
antes-falla,13,ok,6.6,1792817959.666162                    <- última confirmada
antes-falla,14,error,2377.4,1792817962.537133,WTimeoutError
```

```
antes-falla,17,error,4583.9,1792817972.672220,WriteConcernError
despues-falla,18,error,3002.7,1792817975.675176,ServerSelectionTimeoutError
   ... seq 19 a 27: ServerSelectionTimeoutError, unos 3000 ms cada uno ...
despues-falla,28,ok,213.2,1792818002.918506                  <- primera confirmada
```

Los seq 14 a 17 figuran como `antes-falla` porque `docker kill` de dos contenedores tardó varios segundos en devolver: los nodos ya estaban caídos desde ~`...960`, pero la señal se escribió en `...968.94`. Por los tiempos y el tipo de error, esas filas pertenecen a la falla.

Por qué el sobreviviente baja a SECONDARY. Un primario debe ver a una mayoría de votantes (2 de 3). Sin latidos de los otros dos, mongo-3 hace *step down* pasado el `electionTimeoutMillis`, aunque siga vivo. Eso evita el *split-brain*: un primario aislado podría aceptar escrituras que otra mayoría nunca vería. No es una elección, es el primario renunciando; y como un solo nodo no gana ninguna elección, nadie más asume. **Cuánto tardó:** los nodos cayeron entre el seq 13 (`...959,67`) y el envío del seq 14 (`...960,16`); el seq 16 aún recibió `WTimeoutError` en `...968,09` y el seq 17 terminó en `...972,67`, tras lo cual solo hay `ServerSelectionTimeoutError`. El *step down* ocurrió entre unos 8 y 13 s después de la caída, compatible con `electionTimeoutMillis` de 10 s; el CSV no da el instante exacto. **Dos fases de error:** (1) `WTimeoutError` y `WriteConcernError` (seq 14 a 17): mongo-3 todavía era primario, **aplicó** la escritura localmente y no consiguió el *ack* de un segundo nodo dentro de `wtimeout=2000 ms`; el seq 17 terminó como `WriteConcernError` cuando el primario hizo *step down* con la operación esperando. (2) `ServerSelectionTimeoutError` (seq 18 a 27): ya no hay primario y nada llega al servidor.

Error ≠ no escrito (`falla-c-rpo.txt`):

```
intentos 109 · confirmados 95 · con error 14 · presentes 99
PERDIDOS 0
errores que SÍ se escribieron ..... 4  [14, 15, 16, 17]
errores que NO se escribieron ..... 10
```

Los 4 `WTimeoutError`/`WriteConcernError` **sí están** en la base, leídos con `readConcern: majority`: cuando volvieron mongo-1 y mongo-2 replicaron esas entradas del oplog de mongo-3 y quedaron mayoritarias. Los 10 `ServerSelectionTimeoutError` nunca llegaron a un servidor. **Ninguna escritura se confirmó sin mayoría** y el RPO sobre lo confirmado es 0. **Efecto en un cliente que compra:** si `comprar_entradas` recibe un `WTimeoutError`, concluye «no se compró» y reintenta con un **nuevo** `orden_id`, el descuento puede quedar duplicado, porque el primero sí se aplicó. Por eso el RPO se mide sobre lo confirmado (lo que dio error tiene resultado desconocido hasta consultar la base) y el P2 exige operaciones **idempotentes**: el reintento usa el **mismo** `orden_id` (`_id` único, `DuplicateKeyError`, se devuelve el estado existente).

Comparación con la Falla B del Lab 1 (CockroachDB, Raft). En los dos, sin quórum no hay escrituras confirmadas (CP en CAP). En Raft el *leaseholder* aislado tampoco confirma y la petición queda colgada o con timeout; en MongoDB el primario aplica localmente y devuelve `WTimeoutError`, lo que deja escrituras «huérfanas» que luego se replican (o se revierten con *rollback* si otro nodo se hubiera elegido). Para el cliente, los dos casos son ambiguos. **Recuperación** (`falla-c-recovered.txt`): al reiniciar mongo-1 y mongo-2 vuelve la mayoría, mongo-3 gana la elección y queda 1 PRIMARY + 2 SECONDARY. **Contra la expectativa:** todo coincidió: no hubo primario, hubo dos fases de error, 4 errores que sí se escribieron y ninguna escritura confirmada sin mayoría.

5.4. Mayoría 2 de 3

Escenario	Votos vivos	¿Mayoría ($\geq$ 2)?	¿Primario?	¿Escrituras `w:majority`?
Normal	3	sí	sí	sí
A (un secundario caído)	2	sí	sí, el mismo	sí, sin corte
B (primario caído)	2	sí	sí, tras la elección (~10 s)	sí, después del RTO
C (dos caídos)	1	**no**	**no** (*step down*)	**no**

Con 3 nodos se tolera **una** falla. La continuidad de A y de B se debe a que quedan 2 de 3 votos, que siguen siendo mayoría; en C queda 1 y no hay escrituras confirmadas. Un cuarto nodo no aumentaría la tolerancia: la mayoría de 4 es 3 y se seguiría tolerando una sola falla. De ahí el número impar de votantes.